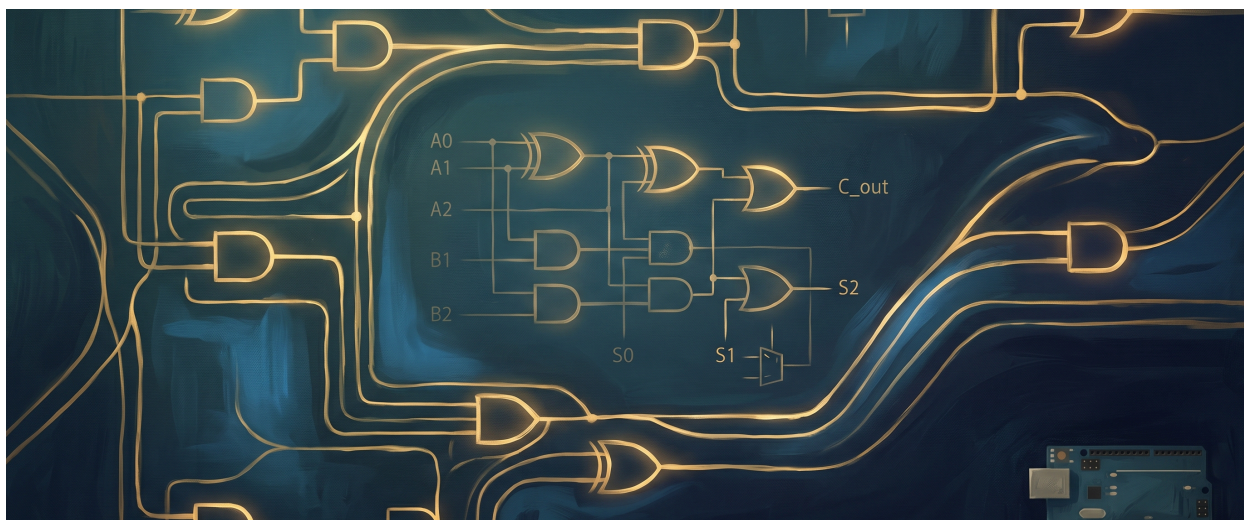




DOCUMENTAÇÃO — ULA Combinacional

Visão geral

Este projeto reúne três blocos lógicos principais:

- um **somador de 3 bits**
- um **complemento de 1 em 2 bits**
- um **comparador de 1 bit**

As entradas usadas no sistema são:

- **Somador:** A0, A1, A2, B0, B1, B2
- **Complemento:** A, B
- **Comparador:** A, B

As saídas são:

- **Somador:** S0, S1, S2, Cout
- **Complemento:** ~A, ~B
- **Comparador:** GT1 = $A \cdot \sim B$ e GT2 = $\sim A \cdot B$

1. Somador de 3 bits

O somador de 3 bits pode ser visto como a ligação em cascata de:

- 1 meio somador no bit menos significativo
- 2 somadores completos nos bits seguintes

Na prática, ele realiza:

```
A2 A1 A0
+ B2 B1 B0
-----
Cout S2 S1 S0
```

1.1 Bit 0

Como o bit 0 não recebe carry de entrada, ele funciona como um meio somador.

Tabela verdade de S0 e C1

A0	B0	S0	C1
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Mapa K de S0

$$S0 = A0 \oplus B0$$

A0 \ B0	0	1
0	0	1
1	1	0

Função simplificada de S0

$$S0 = A0' B0 + A0 B0'$$

Mapa K de C1

$$C1 = A0 \cdot B0$$

A0 \ B0	0	1
0	0	0
1	0	1

Função simplificada de C1

$$C1 = A0B0$$

1.2 Bit 1

O bit 1 recebe **A1**, **B1** e o carry **C1** vindo do bit anterior.

Tabela verdade do somador completo

A1	B1	C1	S1	C2
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Mapa K de S1

$$S1 = A1 \oplus B1 \oplus C1$$

A1 \ B1C1	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Função simplificada de S1

$$S1 = A1 \oplus B1 \oplus C1$$

Forma soma de produtos:

$$S1 = A1'B1'C1 + A1'B1C1' + A1B1'C1' + A1B1C1$$

Mapa K de C2

$$C2 = A1B1 + A1C1 + B1C1$$

A1 \ B1C1	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Função simplificada de C2

$$C2 = A1B1 + A1C1 + B1C1$$

1.3 Bit 2

O bit 2 recebe **A2**, **B2** e o carry **C2**.

Tabela verdade do somador completo

A2	B2	C2	S2	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1

A2	B2	C2	S2	Cout
1	1	0	0	1
1	1	1	1	1

Mapa K de S2

$$S2 = A2 \oplus B2 \oplus C2$$

A2 \ B2C2	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Função simplificada de S2

$$S2 = A2 \oplus B2 \oplus C2$$

Forma soma de produtos:

$$S2 = A2'B2'C2 + A2'B2C2' + A2B2'C2' + A2B2C2$$

Mapa K de Cout

$$Cout = A2B2 + A2C2 + B2C2$$

A2 \ B2C2	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Função simplificada de Cout

$$Cout = A2B2 + A2C2 + B2C2$$

1.4 Equações finais do somador de 3 bits

$$S0 = A0 \oplus B0$$

$$C1 = A0B0$$

$$S1 = A1 \oplus B1 \oplus C1$$

$$C2 = A1B1 + A1C1 + B1C1$$

$$S2 = A2 \oplus B2 \oplus C2$$

$$Cout = A2B2 + A2C2 + B2C2$$

2. Complemento de 1 em 2 bits

O bloco de complemento apenas inverte cada bit de entrada.

Entradas:

- A
- B

Saídas:

- $\sim A$
- $\sim B$

2.1 Tabela verdade

A	B	$\sim A$	$\sim B$
0	0	1	1
0	1	1	0
1	0	0	1
1	1	0	0

2.2 Mapa K de $\sim A$

A \ B	0	1
0	1	1
1	0	0

Função simplificada

$$\sim A = A'$$

2.3 Mapa K de $\sim B$

A \ B	0	1
0	1	0
1	1	0

Função simplificada

$$\sim B = B'$$

3. Comparador de 1 bit

O comparador foi definido por duas saídas:

- $GT1 = A \cdot \sim B \rightarrow$ indica $A > B$
- $GT2 = \sim A \cdot B \rightarrow$ indica $B > A$

Quando as duas saídas são 0, então $A = B$.

3.1 Tabela verdade

A	B	GT1 = $A \cdot \sim B$	GT2 = $\sim A \cdot B$	Relação
0	0	0	0	$A = B$
0	1	0	1	$B > A$
1	0	1	0	$A > B$
1	1	0	0	$A = B$

3.2 Mapa K de GT1

A \ B	0	1
0	0	0
1	1	0

Função simplificada

$$GT1 = A \cdot B'$$

3.3 Mapa K de GT2

A \ B	0	1
0	0	1
1	0	0

Função simplificada

$$GT2 = A' \cdot B$$

4. Resumo final das funções simplificadas

Somador

$$\begin{aligned} S0 &= A0' B0 + A0 B0' \\ C1 &= A0 B0 \\ S1 &= A1 \oplus B1 \oplus C1 \\ C2 &= A1 B1 + A1 C1 + B1 C1 \\ S2 &= A2 \oplus B2 \oplus C2 \\ Cout &= A2 B2 + A2 C2 + B2 C2 \end{aligned}$$

Complemento

$$\sim A = A'$$
$$\sim B = B'$$

Comparador

$$GT1 = A \cdot B'$$
$$GT2 = A' \cdot B$$

5. Uso dos multiplexadores no projeto

Para juntar as saídas das operações e mandar tudo para o Arduino usando as mesmas quatro entradas, foram usados **4 multiplexadores 8×1**.

A ideia é simples: como o resultado final lido pelo Arduino ocupa até 4 bits, cada MUX guarda **um bit de saída** de cada operação. Assim, dependendo do código de operação enviado pelo Arduino, os quatro MUX passam a saída correspondente daquela operação.

A organização ficou assim:

- **MUX 0:** S0, GT1, ~A
- **MUX 1:** S1, GT2, ~B
- **MUX 2:** S2, A
- **MUX 3:** Cout, B

No caso da soma, isso forma o resultado completo em 4 bits:

```
MUX 0 -> S0
MUX 1 -> S1
MUX 2 -> S2
MUX 3 -> Cout
```

No caso do complemento, apenas os dois primeiros MUX são usados com informação útil:

```
MUX 0 -> ~A
MUX 1 -> ~B
```

No caso da comparação, além de indicar a relação lógica entre as entradas, o sistema também retorna os próprios valores originais de **A** e **B**:

```
MUX 0 -> GT1
MUX 1 -> GT2
MUX 2 -> A
MUX 3 -> B
```

Com isso, quando o Arduino envia o serial code da comparação, ele consegue ler:

- se **A > B**
- se **B > A**
- se **A = B**
- e também os valores de **A** e **B** para conferência

A interpretação funciona assim:

- **GT1 = 1** e **GT2 = 0** → **A > B**
- **GT1 = 0** e **GT2 = 1** → **B > A**
- **GT1 = 0** e **GT2 = 0** → **A = B**

Essa abordagem facilita bastante o projeto, porque o Arduino sempre lê o resultado final pelas mesmas entradas, mudando apenas a operação selecionada.

6. Observação final

Ao representar o sistema como uma pequena ULA, as três operações podem ser vistas assim:

- **Somador:** retorna **S0, S1, S2, Cout**
- **Complemento:** retorna **~A, ~B**
- **Comparador:** retorna **GT1, GT2, A, B**

Isso permite selecionar a saída desejada com multiplexadores e com uma lógica de controle por opcode enviada pelo Arduino.

hardware digital e usando software apenas para seleção, leitura e exibição dos resultados.

7. Papel do Arduino no sistema

No projeto, o **Arduino Mega 2560** funciona como a parte de controle, leitura e exibição da ULA.

A lógica principal das operações continua sendo feita no hardware digital, usando o somador, o complemento, o comparador e os multiplexadores. O Arduino entra para:

- enviar o **opcode** da operação para os MUX
- ler os **4 bits finais** vindos dos MUX
- interpretar esses bits dependendo da operação escolhida
- mostrar o resultado em **um display de 7 segmentos em hexadecimal**
- mostrar detalhes no **Monitor Serial**

A ideia geral é:

1. o usuário envia uma sequência binária pelo Serial
 2. o Arduino separa essa sequência em opcodes de 2 bits
 3. o Arduino envia o opcode para os pinos **OP0** e **OP1**
 4. os MUX selecionam as saídas corretas da operação
 5. o Arduino lê os pinos **22, 23, 24 e 25**
 6. o resultado é convertido e mostrado no display hexadecimal
-

8. OpCodes usados

A seleção da operação é feita por 2 bits:

```
00 -> Soma
01 -> Comparação
10 -> Complemento
11 -> Reservado
```

Esses dois bits são enviados pelo Arduino para os MUX através dos pinos:

Sinal	Pino no Arduino	Função
OP0	2	bit menos significativo do opcode
OP1	3	bit mais significativo do opcode

9. Leitura das saídas dos MUX

O Arduino lê sempre os mesmos quatro pinos:

Pino Arduino	Bit lido	Significado geral
22	bit 0	menos significativo
23	bit 1	bit intermediário
24	bit 2	bit intermediário
25	bit 3	mais significativo

A montagem do valor é feita assim:

```
valor = bit0 + 2·bit1 + 4·bit2 + 8·bit3
```

Ou seja:

```
valor = pino22·1 + pino23·2 + pino24·4 + pino25·8
```

10. Organização dos MUX

Foram usados **4 multiplexadores 8×1**, um para cada bit da saída final.

A organização ficou assim:

MUX	Soma	Comparação	Complemento
MUX0	S0	GT1	~A
MUX1	S1	GT2	~B
MUX2	S2	A	—
MUX3	Cout	B	—

Com isso, cada operação usa os mesmos quatro pinos de leitura do Arduino, mas com significados diferentes.

10.1 Soma

Na soma, os quatro MUX formam o resultado completo em 4 bits:

```
[MUX0, MUX1, MUX2, MUX3] = [S0, S1, S2, Cout]
```

O Arduino lê esses quatro bits e mostra o resultado em hexadecimal no display.

Exemplo:

```
0011 -> 3
1010 -> A
1111 -> F
```

10.2 Comparação

Na comparação, os quatro bits lidos são:

```
[MUX0, MUX1, MUX2, MUX3] = [GT1, GT2, A, B]
```

A interpretação é:

```
GT1 = 1 e GT2 = 0 -> A > B
GT1 = 0 e GT2 = 1 -> B > A
GT1 = 0 e GT2 = 0 -> A = B
```

No display, a comparação foi configurada assim:

Condição	Valor mostrado no display
$A > B$	A
$B > A$	b
$A = B$	0

Foi usado **b minúsculo** porque em display de 7 segmentos a letra **B** maiúscula não fica bem representada. O padrão mais legível é usar **b**.

10.3 Complemento

No complemento, os bits que chegam ao Arduino **já são o próprio complemento**.

Então o código não inverte mais os bits.

Ele apenas lê:

```
MUX0 -> ~A
MUX1 -> ~B
```

E transforma isso em decimal/hexadecimal para mostrar no display.

Exemplo:

```
~B~A = 00 -> 0
~B~A = 01 -> 1
~B~A = 10 -> 2
~B~A = 11 -> 3
```

11. Display de 7 segmentos em hexadecimal

No começo do projeto, foram considerados dois displays para mostrar dezena e unidade. Depois, o sistema foi simplificado para usar **apenas um display de 7 segmentos**.

Como a saída final tem 4 bits, o maior valor possível é:

```
11112 = 1510 = F16
```

Por isso, um único display hexadecimal é suficiente para representar qualquer saída de 4 bits.

O display mostra:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, b, C, d, E, F
```

11.1 Pinos dos segmentos

O mapeamento final dos segmentos ficou:

Segmento	Pino Arduino
A	40
B	41
C	42
D	43
E	44
F	45
G	46

Durante os testes, foi verificado que o correto era manter:

```
SEG_B = 41  
SEG_F = 45
```

Com essa configuração, os números passaram a aparecer corretamente no display.

11.2 Pino comum do display

Como está sendo usado apenas um dígito do display, foi usado um único pino comum:

```
DIG_HEX = 47
```

Se for usado o outro lado de um display duplo, basta trocar para outro pino comum, por exemplo:

```
DIG_HEX = 48
```

11.3 Tipo do display

O display usado foi tratado como **ânodo comum**.

Por isso:

- o pino comum do display é ativado em nível alto
- os segmentos acendem em nível baixo

No código, isso ficou separado em duas configurações:

```
const bool COMMON_ANODE = true;  
const bool DIGIT_ACTIVE_HIGH = true;
```

12. Código final do Arduino

```
// ULA com 1 display em hexadecimal  
  
const int OP0 = 2;  
const int OP1 = 3;  
  
const int IN0 = 22;  
const int IN1 = 23;  
const int IN2 = 24;  
const int IN3 = 25;  
  
const bool COMMON_ANODE = true;  
const bool DIGIT_ACTIVE_HIGH = true;  
  
const int SEG_A = 40;  
const int SEG_B = 41;
```



```

const int SEG_C = 42;
const int SEG_D = 43;
const int SEG_E = 44;
const int SEG_F = 45;
const int SEG_G = 46;

const int DIG_HEX = 47;

const int MAX_OPCODES = 64;
uint8_t programOpcodes[MAX_OPCODES];
int programLength = 0;
int pc = 0;

String lineBuffer = "";
int currentValueToDisplay = 0;

struct MuxData {
    int b0;
    int b1;
    int b2;
    int b3;
};

// ordem: a b c d e f g
const bool hexMap[16][7] = {
    {1,1,1,1,1,1,0}, // 0
    {0,1,1,0,0,0,0}, // 1
    {1,1,0,1,1,0,1}, // 2
    {1,1,1,1,0,0,1}, // 3
    {0,1,1,0,0,1,1}, // 4
    {1,0,1,1,0,1,1}, // 5
    {1,0,1,1,1,1,1}, // 6
    {1,1,1,0,0,0,0}, // 7
    {1,1,1,1,1,1,1}, // 8
    {1,1,1,1,0,1,1}, // 9
    {1,1,1,0,1,1,1}, // A

```

```

    {0,0,1,1,1,1,1}, // b
    {1,0,0,1,1,1,0}, // C
    {0,1,1,1,1,0,1}, // d
    {1,0,0,1,1,1,1}, // E
    {1,0,0,0,1,1,1}  // F
};

void writeOpcode(uint8_t opcode) {
    digitalWrite(OP0, opcode & 0x01);
    digitalWrite(OP1, (opcode >> 1) & 0x01);
}

MuxData readInputs() {
    MuxData data;
    data.b0 = digitalRead(IN0);
    data.b1 = digitalRead(IN1);
    data.b2 = digitalRead(IN2);
    data.b3 = digitalRead(IN3);
    return data;
}

int readNibble() {
    MuxData data = readInputs();
    return (data.b0 << 0) | (data.b1 << 1) | (data.b2 << 2) |
    (data.b3 << 3);
}

void printInputBits() {
    MuxData data = readInputs();

    Serial.print("Bits lidos 22 23 24 25 -> ");
    Serial.print(data.b0);
    Serial.print(" ");
    Serial.print(data.b1);
    Serial.print(" ");
    Serial.print(data.b2);
    Serial.print(" ");
    Serial.print(data.b3);
}

```

```

Serial.print(" ");
Serial.println(data.b3);

Serial.print("Binario [25..22] = ");
Serial.print(data.b3);
Serial.print(data.b2);
Serial.print(data.b1);
Serial.println(data.b0);
}

int readSumValue() {
    return readNibble();
}

int readComplementValue() {
    MuxData data = readInputs();

    int y0 = data.b0;
    int y1 = data.b1;

    Serial.print("Complemento recebido bruto: ");
    Serial.print(y1);
    Serial.println(y0);

    return (y0 << 0) | (y1 << 1);
}

int readComparisonDisplayValue() {
    MuxData data = readInputs();

    int gt1 = data.b0;
    int gt2 = data.b1;
    int a    = data.b2;
    int b    = data.b3;

    Serial.print("Comparacao -> GT1=");

```

```

Serial.print(gt1);
Serial.print(" GT2=");
Serial.print(gt2);
Serial.print(" A=");
Serial.print(a);
Serial.print(" B=");
Serial.print(b);
Serial.print(" | Relacao: ");

if (gt1 == 1 && gt2 == 0) {
    Serial.println("A > B");
    return 10;
}

if (gt1 == 0 && gt2 == 1) {
    Serial.println("B > A");
    return 11;
}

if (gt1 == 0 && gt2 == 0) {
    Serial.println("A = B");
    return 0;
}

Serial.println("estado invalido");
return 15;
}

void writeSegmentPin(int pin, bool on) {
    if (COMMON_ANODE) {
        digitalWrite(pin, on ? LOW : HIGH);
    } else {
        digitalWrite(pin, on ? HIGH : LOW);
    }
}

```

```

void setSegmentsForHex(int value) {
    if (value < 0) value = 0;
    if (value > 15) value = 15;

    writeSegmentPin(SEG_A, hexMap[value][0]);
    writeSegmentPin(SEG_B, hexMap[value][1]);
    writeSegmentPin(SEG_C, hexMap[value][2]);
    writeSegmentPin(SEG_D, hexMap[value][3]);
    writeSegmentPin(SEG_E, hexMap[value][4]);
    writeSegmentPin(SEG_F, hexMap[value][5]);
    writeSegmentPin(SEG_G, hexMap[value][6]);
}

void enableDigit(bool on) {
    if (DIGIT_ACTIVE_HIGH) {
        digitalWrite(DIG_HEX, on ? HIGH : LOW);
    } else {
        digitalWrite(DIG_HEX, on ? LOW : HIGH);
    }
}

void turnOffAllSegments() {
    writeSegmentPin(SEG_A, false);
    writeSegmentPin(SEG_B, false);
    writeSegmentPin(SEG_C, false);
    writeSegmentPin(SEG_D, false);
    writeSegmentPin(SEG_E, false);
    writeSegmentPin(SEG_F, false);
    writeSegmentPin(SEG_G, false);
}

void refreshDisplay() {
    int value = currentValueToDisplay;
    if (value < 0) value = 0;
    if (value > 15) value = 15;

```

```

    setSegmentsForHex(value);
    enableDigit(true);
}

bool isBinaryString(const String& s) {
    if (s.length() == 0) return false;

    for (unsigned int i = 0; i < s.length(); i++) {
        if (s[i] != '0' && s[i] != '1') return false;
    }

    return true;
}

void clearProgram() {
    programLength = 0;
    pc = 0;
}

void parseProgramBits(const String& bits) {
    clearProgram();

    int usable = bits.length() - (bits.length() % 2);

    for (int i = 0; i < usable && programLength < MAX_OPCODES;
i += 2) {
        char b1 = bits[i];
        char b0 = bits[i + 1];

        uint8_t opcode = ((b1 == '1') ? 2 : 0) | ((b0 == '1') ? 1
: 0);
        programOpcodes[programLength++] = opcode;
    }
}

void showProgram() {

```

```

Serial.print("Programa carregado: ");

if (programLength == 0) {
    Serial.println("(vazio)");
    return;
}

for (int i = 0; i < programLength; i++) {
    Serial.print((programOpcodes[i] >> 1) & 1);
    Serial.print(programOpcodes[i] & 1);
    Serial.print(' ');
}

Serial.println();
}

void executeNextOpcode() {
    if (programLength == 0) {
        Serial.println("Nenhum programa carregado.");
        return;
    }

    if (pc >= programLength) {
        Serial.println("Fim do programa.");
        return;
    }

    uint8_t opcode = programOpcodes[pc++];
    writeOpcode(opcode);

    delay(20);

    switch (opcode) {
        case 0b00: {
            printInputBits();

```

```

    int value = readSumValue();
    currentValueToDisplay = value;

    Serial.print("OP 00 - SOMA -> decimal ");
    Serial.print(value);
    Serial.print(" | hex ");
    Serial.println(value, HEX);
    break;
}

case 0b01: {
    printInputBits();

    int value = readComparisonDisplayValue();
    currentValueToDisplay = value;

    Serial.print("OP 01 - COMPARACAO -> display ");
    if (value == 10) {
        Serial.println("A");
    } else if (value == 11) {
        Serial.println("b");
    } else if (value == 0) {
        Serial.println("0");
    } else {
        Serial.println("F");
    }
    break;
}

case 0b10: {
    printInputBits();

    int value = readComplementValue();
    currentValueToDisplay = value;

    Serial.print("OP 10 - COMPLEMENTO -> decimal ");

```



```

        Serial.print(value);
        Serial.print(" | hex ");
        Serial.println(value, HEX);
        break;
    }

    case 0b11: {
        printInputBits();

        currentValueToDisplay = 0;
        Serial.println("OP 11 - RESERVADO -> 0");
        break;
    }
}

}

void processLine(String cmd) {
    cmd.trim();

    if (cmd.length() == 0) return;

    if (cmd == "*") {
        executeNextOpcode();
        return;
    }

    if (cmd.equalsIgnoreCase("reset")) {
        clearProgram();
        writeOpcode(0);
        currentValueToDisplay = 0;
        Serial.println("Programa apagado.");
        return;
    }

    if (cmd.equalsIgnoreCase("show")) {
        showProgram();
    }
}

```

```

    return;
}

if (isBinaryString(cmd)) {
    parseProgramBits(cmd);
    Serial.print("Programa carregado com ");
    Serial.print(programLength);
    Serial.println(" opcode(s).");
    showProgram();
    return;
}

Serial.println("Comando invalido.");
}

void setup() {
    Serial.begin(9600);

    pinMode(OP0, OUTPUT);
    pinMode(OP1, OUTPUT);

    pinMode(IN0, INPUT);
    pinMode(IN1, INPUT);
    pinMode(IN2, INPUT);
    pinMode(IN3, INPUT);

    pinMode(SEG_A, OUTPUT);
    pinMode(SEG_B, OUTPUT);
    pinMode(SEG_C, OUTPUT);
    pinMode(SEG_D, OUTPUT);
    pinMode(SEG_E, OUTPUT);
    pinMode(SEG_F, OUTPUT);
    pinMode(SEG_G, OUTPUT);

    pinMode(DIG_HEX, OUTPUT);

```

```

writeOpcode(0);
currentValueToDisplay = 0;

enableDigit(false);
turnOffAllSegments();

Serial.println("=== ULA combinacional - Mega 2560 ===");
Serial.println("Usando 1 display em hexadecimal.");
Serial.println("Comparacao: A>B mostra A, B>A mostra b, iguais mostra 0.");
Serial.println("Complemento: le os bits brutos e converte direto.");
Serial.println("Lendo apenas 22, 23, 24 e 25.");
Serial.println("Envie algo como 000110");
Serial.println("Isso vira: [00] [01] [10]");
Serial.println("Depois envie * para executar uma por vez.");
Serial.println("Comandos:");
Serial.println("*      -> executa a proxima instrucao");
Serial.println("show  -> mostra o programa");
Serial.println("reset -> limpa tudo");
}

void loop() {
    refreshDisplay();

    while (Serial.available() > 0) {
        char c = (char)Serial.read();

        if (c == '\\n' || c == '\\r') {
            if (lineBuffer.length() > 0) {
                processLine(lineBuffer);
                lineBuffer = "";
            }
        } else {
            lineBuffer += c;
        }
    }
}

```

```
}  
}  
}
```

13. Explicação do código

13.1 Escrita do opcode

A função `writeOpcode()` envia o opcode para os pinos `OP0` e `OP1`.

Esses dois sinais vão para todos os MUX e escolhem qual operação será exibida na saída.

13.2 Leitura dos bits finais

A função `readInputs()` lê os pinos `22`, `23`, `24` e `25`.

A função `readNibble()` transforma esses quatro bits em um valor numérico de 0 a 15.

13.3 Soma

Na soma, o Arduino apenas lê os quatro bits e mostra o valor hexadecimal correspondente.

```
Soma -> [S0, S1, S2, Cout]
```

13.4 Comparação

Na comparação, o Arduino lê `GT1`, `GT2`, `A` e `B`.

Depois mostra no display:

```
A > B -> A  
B > A -> b  
A = B -> 0
```

13.5 Complemento

No complemento, os bits recebidos já são os bits complementados.

Por isso, o código não usa `!` para inverter.

Ele apenas faz:

```
valor = ~A + 2*~B
```

E mostra esse valor em hexadecimal.

13.6 Display hexadecimal

A tabela `hexMap` define quais segmentos acendem para cada valor.

A ordem usada é:

```
A, B, C, D, E, F, G
```

Como o display é de ânodo comum, a função `writeSegmentPin()` inverte a lógica automaticamente:

```
segmento ligado -> LOW  
segmento desligado -> HIGH
```

14. Resultado final

Com as mudanças feitas, o sistema ficou assim:

- o Arduino lê apenas os pinos `22, 23, 24 e 25`
- a saída é exibida em apenas **um display de 7 segmentos**
- o valor é mostrado em **hexadecimal**
- a soma pode mostrar de `0` até `F`
- a comparação mostra `A`, `b` ou `0`
- o complemento usa os bits brutos já complementados
- os segmentos finais ficaram com `B = 41` e `F = 45`

Esse formato simplificou a parte visual do projeto, porque um único display hexadecimal consegue representar qualquer resultado de 4 bits.
